

Process fusion: An industrial case study on agile software product line engineering

Geir K. Hanssen ^{a,b,*}, Tor E. Fægri ^a

^a SINTEF ICT, NO-7465 Trondheim, Norway

^b Norwegian University of Science and Technology, Department of Computer and Information Science, NO-7491 Trondheim, Norway

Received 27 March 2007; received in revised form 19 July 2007; accepted 10 October 2007

Available online 7 November 2007

Abstract

This paper presents a case study of a software product company that has successfully integrated practices from software product line engineering and agile software development. We show how practices from the two fields support the company's strategic and tactical ambitions, respectively. We also discuss how the company integrates strategic, tactical and operational processes to optimize collaboration and consequently improve its ability to meet market needs, opportunities and challenges. The findings from this study are relevant to software product companies seeking ways to balance agility and product management. The findings also contribute to research on industrializing software engineering.

© 2007 Elsevier Inc. All rights reserved.

Keywords: Software product development; Software product management; Software product line engineering; Agile software development

1. Introduction

Software engineering is a complex task that involves fast-paced product development, competitive market conditions, knowledge management, organizational factors, rapidly evolving technologies, etc. As a means of handling this complexity, establishing control and using resources efficiently, development organizations normally adopt a method that defines how their software engineering activities should be carried out within the given context. A method may cover a broad range of issues; typically estimation, design, and development, among others. A great many methodologies are available and they vary a lot; from strict plan-based approaches to agile approaches and any variant in between. Regardless of the method being used, most software projects strive to reach a balance between

three basic goals: satisfactory software quality (scope), the right cost and timely delivery. Attempting to satisfy these requirements causes further complications to arise, particularly with respect to long-term product management issues.

This paper discusses two popular development approaches; software product line engineering (SPLE) and agile software development (ASD). It describes how these have been combined to improve the ability to achieve the three basic goals. These two approaches can, in their most radical forms, be placed in each end of a plan-based/agile spectrum (Boehm, 2002). The former is based on planning and preparations for efficient software development based on rapid construction by assembling predeveloped assets, while the latter aims at efficient change response instead of extensive up-front planning. They may correspondingly be categorized as proactive and reactive approaches to software engineering. The principles of SPLE have been in use for a long time and industrial experience shows that the approach has substantial advantages when it comes to cost-efficient development, product quality and the ability

* Corresponding author. Address: SINTEF ICT, NO-7465 Trondheim, Norway.

E-mail addresses: ghanssen@sintef.no (G.K. Hanssen), tor.e.fegri@sintef.no (T.E. Fægri).

to deliver on time (Birk et al., 2003; Linden, 2002; Bosch, 2000; Clements and Northrop, 2002). ASD is a more recent trend within the software engineering community and has attracted wide interest in both industry and academia. Initial evidence from the results of ongoing research suggests that using ASD is advantageous, given the right context (Boehm, 2002; Erickson et al., 2005). Both approaches have the same overall objective; that of improving software development efficiency.

Recently, there has been interest in investigating whether, and if so how, SPLE and ASD can be combined to complement each other (Carbon et al., 2006; Tian and Cooper, 2006), but little empirical research on the topic has been conducted. In this paper, we present our findings from a case study of a medium-sized, Norwegian software product company called CompNN (real name made anonymous). CompNN has used and matured the ASD method Evo, short for *Evolutionary Project Management* (Gilb, 2005) for three years, covering four releases of their main product.

Our study is based on a well established industry–researcher relationship that began several years ago. The study expands upon a previous one, in which we investigated the introduction and preliminary effects of Evo (Hanssen and Fægri, 2006). This previous study gave us insights into several important aspects of the use of an agile process in a product development context, with respect to benefits, problems and prerequisites.

The aim of the study reported herein was to describe and analyze an industrial case to understand how SPLE and ASD can be combined and to clarify associated costs and gains.

Next, we provide an overview of SPLE and ASD and explain how they both relate to, and conflict with, each other (Section 2). Then we explain the context of the case study (Section 3). Then, we present the study method (Section 4). Subsequently, we present the findings from the study (Section 5), discuss the results (Section 6) and offer our conclusions (Section 7).

2. State of the practice

SPLE is an approach to software engineering in which a set of products is built on a common set of core assets (Bosch, 2000; Clements and Northrop, 2002). SPLE draws upon principles similar to those that have been used for decades in most other industries, in which production includes the assembly of prefabricated parts. SPLE is claimed to have numerous benefits: large-scale productivity gains, decreased time-to-market, increased product quality, increased customer satisfaction, more efficient use of human resources, and improved ability to apply mass customization, maintain market presence and sustain unprecedented growth. However, achieving these claimed benefits requires careful attention to a wide range of issues (Linden, 2002; Knauber et al., 2000; Käkölä and Dueñas, 2006).

The basic assumption underpinning SPLE's claimed advantages is that there is sufficient commonality among multiple products to justify investment in their systematic reuse. Reusable assets, of various kinds, are said to constitute the *product line platform*. Examples include requirements, architectures, designs, code and test cases. Domain engineering consists of the set of activities necessary to define, extend and maintain the platform. The platform, the key to an SPLE approach, bears the cost of domain engineering, which can, presumably, be amortized across multiple, derived products.¹

Multiple preconditions must be satisfied if an SPLE approach is to be profitable, but the most fundamental of these is that the market for the products be predicted accurately. If prediction of market trends is inaccurate, SPLE will *inhibit* the business potential of the organization by enforcing wasteful effort related to adherence to reference architectures, organizational overhead, reuse strategies, etc., essentially slowing down the software production. In addition, product management is more complicated due to complex dependencies between product variants and the scope of the platform (Bosch, 2006; Helferich et al., 2006). SPLE has an influence on most practices in software engineering in an organization. Thus, SPLE activities must be based on the strategic ambitions of the organization and, in the service of such ambitions, careful planning. SPLE demands long-term investment and effort. Nevertheless, given the right processes and a suitable product platform, new products can be derived faster, at lower cost, and with more predictable quality (Birk et al., 2003). SPLE has already become economically necessary for some companies (Linden, 2002).

There are more or less rigorous ways to apply SPLE practices. For example, product derivation can be implemented using proactive engineering or reactive engineering. In a proactive approach, the reusable assets are developed before the product is built. In a reactive approach, the product is built first and subsequently the reusable assets are recovered from the product and integrated back into the platform. Further, variability models are used to represent variation among the products that the platform should support, i.e. which variation defines the scope of the product line? A vast number of different techniques and supportive artifacts exist for doing SPLE, but their discussion lies beyond the scope of this paper. For further details, please refer to, e.g. (Bosch, 2000; Clements and Northrop, 2002). It is useful to consider SPLE as a spectrum of approaches to software engineering.² SPLE may have a wide impact on business, organization, process and technological factors (Sugumaran et al., 2006). Therefore, organizations that adopt SPLE must choose their own path by introducing a set of practices that suit the

¹ We say that a product is *derived* from the platform if a significant part of the product is reused from it.

² <http://www.sei.cmu.edu/productlines/framework.html> (accessed December 2006).

objectives and capabilities of the organization (Lorge Parnas and Clements, 1986). Naturally, as with any other innovation in software processes, SPLE practices should be reviewed and revised continually to ensure learning and improvement.

Agile software development (ASD) lies at the other end of the plan-based/agile spectrum. ASD focuses little on planning and expects limited predictability. Further, it does not address issues of reuse. Hence, it contrasts sharply with the plan-centric SPLE approach. While SPLE supports the strategic objectives of the organization, ASD primarily benefits tactical ambitions, for example, addressing the immediate demands of customers or targeting smaller-scale development efforts.

ASD is a common name for a set of defined methods for software development. Some of the best known and used are Extreme Programming (Beck, 2000) and Scrum (Schwaber and Beedle, 2001). Although these agile methods vary in focus and presentation, they are all based on a few simple guidelines for ASD, defined by the *Agile Manifesto*³ which states four basic values: (1) *Individuals and interactions over processes and tools*; ASD emphasize self-organizing teams, close cooperation with the user(s) and informal and direct communication. (2) *Working software over comprehensive documentation*; the main goal, and main proof of progress, is working software, not models or demonstrators. Documentation is kept on a strictly need-to-have basis. (3) *Customer collaboration over contract negotiation*; the customer is given an active role in development and thus has a direct responsibility to state requirements and correspondingly to verify both incremental and final results. (4) *Responding to change over following a plan*; development is a dynamic and creative endeavor, so plans are kept on a minimum with the focus being on flexibility and response to change. In addition to these four values, the Agile Manifesto also defines a handful of principles that form rules for development. These again are used to form a set of practices in the various agile methods. Some methods emphasize practice and technique descriptions (such as Extreme Programming), while others place more emphasis on management activities (such as Scrum) (Abrahamsson et al., 2002).

As part of the interest in, and uptake of, agile methods in industry a variety of experiences have been reported. Most of these can be defined as simple, single-case studies, but there are some reports that have investigated more deeply and have contributed to a growing knowledge base on the costs and benefits of agile methods. The agile method Extreme Programming has attracted the most attention. Within this method, the practice of pair programming has been the most investigated by far (Erickson et al., 2005). In this regard, a recent large controlled experiment has clarified the conditions under which pairs of developers perform better than individuals and vice versa (Arisholm

et al., 2007). Task complexity and level of experience were shown to influence the outcome. Another relatively well-investigated agile practice is test-driven development (TDD) (Müller and Hagner, 2002; George and Williams, 2004; Erdogmus and Morisio, 2005). The results are somewhat contradictory. Erdogmus and Morisio (2005) showed that TDD improves productivity but not quality, yet their results also indicated that TDD programmers achieve more consistent quality results. With respect to Extreme Programming as a whole, Erickson et al. (2005) has reviewed numerous studies on the method and found no clear evidence as to its benefits. A possible reason is that Extreme Programming, as along with other agile methods, is constituted by a set of diverse guidelines, each of which should be investigated individually.

SPLE and ASD have significant differences due to the fundamentally different challenges they are intended to solve. SPLE addresses longer-term strategic objectives related to life-cycle product management, whereas ASD addresses short-term tactical objectives, such as single projects developing one specific product. In practice, an organization will face both strategic and tactical challenges. Thus, it makes sense to investigate whether, and if so how, these two approaches can complement each other. The table below summarizes how the SPLE approach corresponds to the twelve principles of the Agile Manifesto (see Table 1).

This comparison shows that ASD and SPLE have few conflicts. They are virtually independent of each other. On most counts, they possess complimentary properties (Carbon et al., 2006; Tian and Cooper, 2006). Whereas SPLE attempts to introduce large-scale cross-product economic benefits through reuse, ASD focuses on delivering single products with the best possible quality for the customer. Thus, there is, *prima facie*, no reason why these two approaches should not be combined. On the contrary, there is a clear motivation to combine them.

SPLE as such is agnostic regarding processes of software development. One can use product line engineering with different software process models. However, intuitively, there are a number of interesting issues to be addressed when considering the use of agile methods in a SPLE setting:

- SPLE advocates a conscious adherence to architecture. Software reuse beyond the most trivial level demands that numerous constraints and guidelines be respected regarding both the reused asset and the consumer of the asset. Agile methods have a reputation for paying little attention to architecture, and occasionally being downright damaging to it. Naturally, architecture erosion will result if constraints and guidelines are broken. Hence, it is useful to discuss the practices used by agile product companies that enable long-term architecture conformity and prosperity of the product line.
- Agile methods are (arguably) best suited to projects of modest to moderate size (Boehm and Turner, 2004). The different products (or variants) in a product line

³ www.agilemanifesto.org (accessed November 2006).

Table 1
Comparing ASD and SPLE

Agile principle	SPLE correspondence
Priority on early and continuous delivery of valuable software	Dependent upon SPLE approach. A reactive model supports early delivery of valuable software (no particular bias with respect to continuous delivery)
Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage	Foreseen changes covered by platform and variability models are cheap. Other changes become increasingly expensive closer to domain engineering artifacts as they may participate as elements in multiple products, and thus increase the cost of maintenance
Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale	An SPLE approach does not reject or inhibit this principle. Product delivery is, nevertheless, somewhat slowed down compared to agile development, due to overhead in maintaining integrity with the product platform
Business people and developers must work together daily throughout the project	An SPLE approach neither rejects nor inhibits this practice
Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done	At the product level, an SPLE approach neither rejects nor inhibits this practice. At the platform level, however, more formalism is required
The most efficient and effective method of conveying information to and within a development team is face-to-face conversation	SPLE, due to increased technical and/or organizational complexity, introduces a need for more explicit, formal and disciplined communication
Working software is the primary measure of progress	SPLE incorporates significant value in the platform that is more difficult to measure. However, for product development, an SPLE approach neither rejects nor inhibits this principle
Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely	SPLE promotes sustainable development at a larger scale
Continuous attention to technical excellence and good design enhances agility	SPLE platforms encode technical excellence and design to support rapid product derivation
Simplicity – the art of maximizing the amount of work not done – is – essential	The main motivation for SPLE is the same – but by using reuse as vehicle for eliminating work. In a reactive SPLE approach, the risk of doing unnecessary work is reduced
The best architectures, requirements, and designs emerge from self-organizing teams	The assumption of SPLE is that investments in requirements, architectures, and designs can be reused successfully across multiple products
At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly	An SPLE approach neither rejects nor inhibits this practice. In fact, due to investments in the platform, most SPLE organization promotes experience gathering and learning

can be considered as semi-autonomous projects within the line. In this regard, SPLE may be considered as a way of partitioning the work of software product companies and subsequently assisting the use of agile methods in larger-scale development efforts.

How can potential benefits and challenges be managed in practice?

3. Study context

CompNN is a medium-sized Norwegian software company that develops, maintains and markets a product line called ProdNN (real name made anonymous). The products derived from the product line are aimed at the high-end market of market and customer surveys. CompNN has a wide customer base that includes some of the world's largest market research agencies. The company was established by three friends in 1996 and has since grown steadily, such that it now has about 90 employees and offices in Norway, the UK, Vietnam, and the USA. There has been a gradual shift from building custom-made applications to a product line. Today, licenses for the ProdNN products constitute most of the revenue.

The bulk of research and development is done at the main office in Norway supported by developers in Viet-

nam. The other offices are mainly devoted to marketing, sales, and customer support. However, as we will show later, they have a clear role to play in providing important input to the development process. Physical presence near the largest markets abroad is important both to serve customers and to include them in the development of the product.

ProdNN can be defined as a single product, but there are many ways to use it. The system is modular. It has five main modules (with numerous submodules): *authoring* to assist the planning of surveys, *panel* to support panel surveys, a *survey engine* that serves most use cases, *reporting* to produce survey reports, and *data transfer* to feed the database for analysis. The use of these modules varies according to the use case. Some modules are central and always in use, while the use of the others depends on the situation. As CompNN's customers vary greatly in size and operation, the ProdNN tool is built to be customizable. CompNN operates with a set of predefined configurations for the most common use cases, but there is also built-in support for detailed customization to support more variants. However, in some cases, a customer has needs that go beyond the functionality available in the product platform. In these cases, CompNN may provide a custom-developed solution for the customer, with the proviso that the result may be included in the platform. If this is

not acceptable to the customer CompNN will deny the request. This strategy of restrictive scoping is chosen to avoid too heavy a burden with respect to single-case development, because that would hamper the continuous development of the platform.

The adoption of Evo was a deliberate choice to move on from an inefficient waterfall-like process (Fægri and Hanssen, 2007). In this case, SPLE was not introduced deliberately, as prescribed by common guidelines, e.g. the Software Engineering Institute's guidelines.⁴ However, this company's development process clearly coincides with some of the fundamental principles of SPLE. Thus, we were able to use this case to understand how the company has combined SPLE and ASD and to identify the experienced effects of the combination. We believe that this is valuable input to the process of shaping an agile product line engineering practice, which is in its early stages.

Several roles cooperate at different levels in the development of the product platform. At a tactical level, the developers in the R&D department handle the software development, guided by the Evo process. The developers are organized in small teams, each of which has three to five persons. To ensure that focus is maintained, each team is usually dedicated to the development of just one of the main modules in the platform. In addition, each team may also cooperate directly with a selected stakeholder. The R&D department has a Chief Technical Officer (CTO), who is responsible for the platform architecture and for ensuring technical alignment with the business domain, and who oversees overall product planning. CompNN puts great emphasis on new and improved qualities which leads to a high pace in the development projects and an unsatisfactory error density in released products. This issue is now being addressed by the adoption of test-driven development.

To handle the diverse processes towards the market at both strategic and tactical levels, CompNN has a Product Management Team (PMT). This function was established at the same time as Evo was introduced, in 2004. The PMT has six members, one of whom acts as the PMT manager. The PMT plays an important role at several levels. They are responsible for marketing initiatives, sales, and key account management. The product roadmap, together with supporting business cases, are the main assets developed by the PMT. Two main channels of information are used to develop these. First, there is the PMT member's knowledge of the domain, market and specific customers. Secondly, the ProdNN Advisory Board, was established recently. This is a group of the most important customers, who provide their ideas and state their requirements for future development. The members of this forum are CEOs (Chief Executive Officer), board members and other representatives from top management.

One of the core principles in the adopted Evo process is the close cooperation with selected stakeholders (selected representatives of important requirements). When a product roadmap has been approved by the executive management, the PMT appoints stakeholders, who agree to participate actively in the release projects. Usually, one stakeholder per project is appointed, which again places the focus on one of the modules or a well-defined feature (Fægri and Hanssen, 2007). The criterion for appointing a stakeholder is that the development focus or goal of the project addresses a module or feature in which the stakeholder has a special interest, which in turn provides sufficient motivation for investing resources in development. In some cases, internal stakeholders are appointed, typically when the project addresses issues that are not directly visible to the end users. External stakeholders are not given any compensation for participating. The sole reason for cooperating is the opportunity to affect and direct the development.

The roles and their participation in product development activities are summarized in Fig. 1. Their interplay is further elaborated in Section 5.

4. Study method

4.1. Rationale

As this study sought to investigate the diverse and complex topic of the fusion of product-line engineering and agile software development, which touches on a variety of factors, such as technology, architecture, process, market development, innovation, and management and organizational issues, we chose a qualitative approach. Our aim was to present a broad view and to explain how this fusion has worked in practice in actual industry, with a view to understanding how SPLE and ASD can be adopted and used to complement each other. We believed that we needed to understand the practicalities and opportunities of this fusion better before we can plan more specific studies on, for example, the effect on time, cost and quality. Thus, we performed a single-case study (Yin and Campbell, 2002) to obtain detailed knowledge about a complete product development organization to be able to understand how internal and external roles interplay at various levels.

4.2. Data collection

Our main source of data collection was a series of interviews. We used existing interviews from a previous study (Hanssen and Fægri, 2006). There were nine interviews in total, which three roles: three customers (different companies) from the development of a single release (three interviews), all members of a product management team (five interviews) and a group of six developers (group interview) where all of these had recent experience from Evo-projects. To acquire a deeper understanding of the strategic and

⁴ http://www.sei.cmu.edu/productlines/adopting_spl.html (accessed December 2006).

Process level	Activities	Roles									Artifacts
		PMT	CTO	CEO	Adv. board	Proj. mgmt.	R&D	Stakeholders	Users	Support	
Strategic	Product planning	X	X	X	X					X	Feedback ← Market knowledge Roadmap Business case
	Release development	X	X			X	X	X			Roadmap ←
Tactical	Proj. Management					X	X				IET Software ←
	Software use										Software ←
Operational	Customer support	X							X	X	Feedback ← Change req.

Fig. 1. Roles, activities and artifacts.

tactical processes and how the development process and the product line system architecture relate to each other, we extended the interview material with thorough semi structured interviews with the PMT manager and the CTO. These two roles are central to the planning and management of the product development process. We defined an interview guide for each of the PMT manager and CTO, but the interviews were semi-structured so that additional information, which was not originally covered in the interview guides, could be acquired. All interviews except the group interview with the developers were done via telephone. Furthermore, all interviews were recorded using a Dictaphone. In sum, we thus base our case study on eleven interviews covering all roles involved in the product line engineering and the applied agile process.

As a secondary data source, we used documents such as product roadmaps and business cases, which are important assets in long- and short-term development of the product line. These documents were used to understand the overall practice of software development and market-related activities and how these relate to each other.

4.3. Data analysis

All recorded interviews were transcribed to form a basis for a thorough and detailed textual analysis. In cases of interviews that covered more than one individual per role, for example the members of the product management team, we analyzed the material according to the principles of constant comparison (Seaman, 1999) using the NVivo™ tool for textual encoding. The transcribed interviews and the documents were used to make a description of the product development (see Section 5).

4.4. Bias and limitations

Several factors restrict the credibility and generalizability of the study. (1) We collected and analyzed data from only one company. However, if we had involved more company cases, the study would have been much more difficult to perform. In addition, we only know of, and have

access to, one company that has adopted and combined both SPLE and ASD practices; the one that we studied. Despite this limitation, a reader may, by considering the context information, be able to identify experiences that may be transferred to another software development organization. (2) There is a threat with respect to the completeness of our study. We did not interview all parties involved in the total process of developing the product platform, but we believe we have selected the most central; those that could give us a good basis for reflection. To strengthen the study further, we incorporated the analysis of important documentation.

5. Findings – a holistic view of software engineering at CompNN

We now describe, from a holistic point of view, how CompNN develops and maintains their product platform. We have identified three distinct software processes and describe how they interact. Then we describe the product platform architecture and related services followed by a description of how the organization has shifted to this new development regime.

5.1. A symbiosis of three interacting customer-centric software processes

Organizations of some size need well defined structure and operational formalism to ensure that important functions are implemented and maintained in a professional manner. CompNN has matured their approach to software engineering over the last 10 years. As new challenges have appeared, CompNN has responded by making different adjustments and innovations in their processes. Because CompNN operates in a highly competitive market, they must continually look for opportunities to improve the way they do business. Thus, the description presented here is merely a snapshot of how things are done currently.

The Fig. 2 illustrates how CompNN, through a symbiosis of three different software processes, connects software development and customer collaboration. The embedded

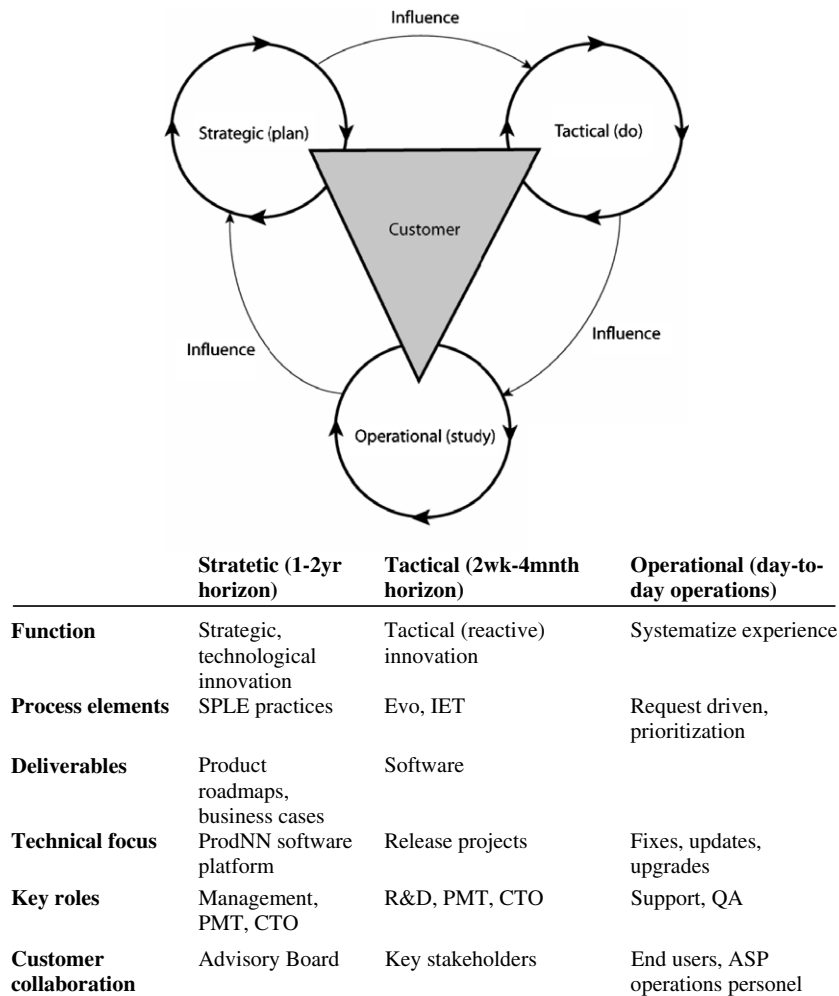


Fig. 2. A customer-centric product development process.

table highlights the various issues of particular attention in the three processes. As a result, CompNN is able to embrace the market all the way down to individual users, and thus improve their ability to respond and interact with their customers. We believe this works well because the three processes serve different but important needs:

Strategic (long-term, SPLE): Evolution at the SPLE level addresses the need to implement long-term strategic plans. SPLE supports CompNN's need to introduce tactical innovation, which is fuelled, for example, by innovations in technology, changes in the business models, or new business opportunities. ProdNN is a product platform that contains a set of building blocks that can be tailored to provide different product offerings (ProdNN solutions). This platform can be considered as the vehicle used by the CTO and PMT to implement long-term visions. More often than not, these innovations are costly in terms of effort and risk. Thus, they should be planned and introduced with great care, in order to exploit the organization's development capability. SPLE-style development inherently supports this ambition.

Tactical (medium-term, ASD): Evolution at the project level (note that a release typically includes 4–6 projects)

addresses the need to embrace customers and listen to their more down-to-earth needs. Close cooperation ensures that a customer's ideas and experience with the software is translated into what we may call tactical innovation, i.e. reactive innovation that seeks to polish, improve, adjust or otherwise make small to moderate adjustments to the product. As the customer's requirements mature during the project, ASD-style development with moderate-size projects works well to accomplish this aim.

Operational (short-term, day-to-day operations): Customers rely on the software to help them to implement vital business functions in their own organizations. Thus, it is of great importance that CompNN is able to sustain a good level of satisfaction with the software in its day-to-day use. During operation, the products are exposed to more users, which exposure increases the potential amount of feedback to the innovations implemented at the strategic and tactical levels.

These three processes are concurrent. They overlap in terms of activities, people and artifacts. All three embrace the customers and give CompNN the opportunity to build strong, cross-level, collaborative relationships with them.

5.1.1. *The strategic process elaborated*

The market process is at a strategic level and provides vital input to the work on the biannual releases and the development projects. This strategic work is operated by the product management team (PMT), who are engaged both in product development and other general market-oriented activities, such as sales, marketing, and key account management. Thus, they act as a link between market (strategic) and development (tactical) activities.

The central activity in the market process is the development of a roadmap. This is a strategic document where needs, ideas and opportunities for future development are documented. In practice, the roadmap is a set of PowerPoint slides that defines growth opportunities, main challenges and strategies. Each issue is described at a high level, to prevent the view of features and functionality from becoming too detailed. On the basis of the suggested direction for developing the product platform, the PMT also indicates how existing or new resources should be used; for example, a need to extend the R&D department. The roadmap also defines what is called ‘Commitments and must haves’, which is a high-level list of improvements and extensions that presumably, will meet the identified opportunities and challenges. To operationalize the roadmap, the PMT also suggests how to set up a handful of development projects to reach the defined commitments and must haves. This includes the development focus per project and staffing (number of developers per project). To establish a long-term view on development, e.g. to expose resource needs, this part of the roadmap characterizes the release projects for both upcoming and subsequent releases, though subsequent releases are characterized more loosely. The last part of the roadmap looks even further into the future and defines potential new areas for the following two years. This overview is developed further in the following year’s roadmap.

The PMT develops a new roadmap once a year. A proposal is given to executive management in the autumn for approval, and is then used as a guide to decide formally which parts of the product platform will be addressed in the upcoming releases. The goal descriptions in the roadmap are deliberately kept at a high-level. Thus, the possibility is left open that projects may suggest refinements to the platform to facilitate achieving the goals. This is important, because the development projects will utilize core agile principles, such as short iterations and close customer engagement as defined by the Evo development process, to capture detailed requirements for finding the best possible practical solution for achieving the high-level goals defined in the roadmap.

The roadmap relates to a set of business cases, which are documents that illustrate the assumed effect of achieving the high-level goals. A business case usually addresses one of the modules in the product platform to obtain a manageable focus and follows a standard structure: it defines the stakeholders, the scope of the project (what is to be done), and its rationale. A business case also contains

a discussion on the return on investment, identified risks and threats, and associated costs.

To provide management with the best possible foundation for making their decisions what is proposed in the roadmap, it also contains descriptions of cases that are not a part of it, thus illustrating trade-offs that must be made by executive management due to limitations of time and resources; the total quantity of ideas always exceeds the quantity that can be developed for one release. The PMT thus keep track of ideas that are excluded from the roadmap and these are revised each time a new roadmap is developed.

5.1.2. *The tactical process elaborated*

Gilb (2005) is the agile method that CompNN adopted at the tactical level, i.e. the level where software is developed. It is not as well-known as Extreme Programming (Beck, 2000) and Scrum (Schwaber and Beedle, 2001), yet it is clearly an agile method, because it is customer-centric and strongly emphasizes short iterations and the frequent delivery of working software. In Evo, the product evolves iteratively as new requirements are discovered, constructed into the code, and subsequently tested. Stakeholders (customers or internal) are expected to give feedback on each of the iterations of the product in order to guide the development of the next iteration. Finally, suites of integration tests are performed before the final system test. Evo is a rigorous process that draws upon principles from the risk-driven Spiral model (Boehm, 1988) but is ultimately driven by the highest possible stakeholder value in each iteration, rather than by risk reduction.

The core process of Evo is requirements management, which serves two purposes per iteration. Firstly, selected stakeholders evaluate the achievements in the previous iteration in terms of a hands-on test or some other type of review. This helps to ensure that progress is in line with stakeholders’ expectations. Secondly, on the basis of stakeholders’ feedback, it is decided what existing requirements need to be improved and what new requirements should be targeted for the next iteration. Requirements are not specified as functions or features, but as product *goals* that address real business concerns of the customer business concerns. Great emphasis is placed on identifying metrics and associating them with each goal. CompNN uses Impact Estimation Tables (IET) as its key project management tool. Each of the projects within a release has an IET, which is essentially a spreadsheet that lists goals vertically and project iterations horizontally. Each of the iterations is broken down into one or two solutions (design proposals) that are believed to address the goals. The structure helps to separate requirements from solutions. By documenting both requirements and project progress (measured results) per iteration, the IET becomes both a plan for, and historical record of, the project. Before release of the iteration is initiated (which initiation spans roughly two weeks), selected customers are invited to give input to product plans with initial goals for the release. During initiation,

the most promising ways of addressing the initial goals are noted and laid out for a number of future iterations in the IET. CompNN's CTO explained: "This plan helps us ensure adherence to an overall technical strategy for product development". Evo at CompNN is a tightly scheduled process in which biweekly iterations follow a fixed schedule (called the 'Evo-week') that defines each role's responsibilities per day.

In addition to playing a vital role in the strategic process, the PMT also have responsibilities in the tactical process, in connection with the software development in the release projects. Because they have the best access to knowledge of the market and the customers (Keil and Carmel, 1995), the PMT are responsible for appointing customer stakeholders in the development projects and for maintaining the link between CompNN and these stakeholders. If a customer stakeholder leaves a project for any reason, it is vital that the PMT take action to appoint another stakeholder or to fill the gap in some other way.

During the second Evo-release, CompNN introduces an additional concept: the green week. This is a week dedicated to error correction and improvements, without engaging with customers directly.

Another great aid in the development process (while not being a part of Evo) is the Continuous Integration framework (CI). This is a set of well-integrated tools for code management, automated tests and builds that enable swift production of product versions ready for testing by stakeholders.

5.1.3. *The operational process elaborated*

The operational process consists of the day-to-day activities in connection with the customer's use of the products based on the product platform. As CompNN provides up to two releases per year, which surpasses the update frequency of most customers, there are several versions of the platform in use at any one time. In addition, there are many variants based on the product platform in use, all of which may be considered advanced applications. In sum, this means that there is great variance in the support requests to CompNN, which are handled by a dedicated support department. Depending on the situation, support may involve guidance in use of the product, scripting, programming, quality assurance of customers' use, or issues related to maintenance. Some requests also address errors and flaws in the software, even ideas for future development. Such requests are recorded in a support system and are used as input to the strategic process that is aimed at upcoming releases. In addition to assisting in the use of the software and recording issues that pertain to potential improvement, the support department develops user documentation.

5.2. *Product platform architecture and services*

The ProdNN architecture is deliberately kept simple to achieve flexibility. From a technical point of view, it is a

traditional three-tiered application with a data layer, a business layer and a web-based (ASP) interface layer. The whole platform is based on the Microsoft.Net platform. Some old COM-components are present and constitute a legacy from the first years of shaping the platform. Recently, the R&D department did so-called interim releases, in which no external stakeholders were involved, and concerning which the only purpose was to port old code, fix errors and resolve pressing open issues. Now, these types of task are performed using free development resources in between the ordinary releases.

All end-user interaction with the software is done via a cross-platform web-based interface and two alternative delivery models; for sophisticated use or large organizations, CompNN offers ProdNN as a stand-alone server solution, which allows the customer to operate the server locally. For less demanding users, CompNN offers an ASP alternative, in which the application is hosted by CompNN.

From a functional point of view, the platform consists of five main modules (with submodules), each of which supports specific features or tasks in the survey process. Some modules are essential in all use cases, while others are optional and depend on the situation. Each module is designed to be configurable to achieve the benefit of developing one common component that can serve many situations of use. Variability in customers use is handled simply; a license file is defined for each customer, which file determines the modules that the customer can use. This solution is the same for both the local server and web-access variants. Working on the next release, development is usually organized such that one development team (defined as a project) is responsible for one of these modules. R&D will, in the future, establish dedicated teams for the modules, which teams will specialize in the features and the related sub domain. ProdNN may be extended by a customer or a provider of third-party solution providers by the use of a set of API's (offered as web services), there being one library for each of the five modules.

ProdNN is not provided as a single standard product, but is defined as a product platform that forms the basis for many variants. As a part of this paradigm, CompNN offers a set of services to assist each customer in deciding upon a variant that matches their specific needs in each case. In most cases, this is done through the 'customized solution' service, where the basic modules are composed and adjusted. This service may also include documentation, training, support and maintenance. Examples of customized solutions in use are help-desk quality feedback systems and claim management systems. In cases where a customer has requirements that are not supported by the product platform, CompNN offers the 'custom development' service which, besides customization, also involves the development of new features. This service may be offered if the new features under consideration are thought to be reusable in a future version of the product platform. It is an absolute requirement that the customer agrees to allow

the new features to become available to other customers as part of the product platform. If this is not feasible for the customer, CompNN will not develop the features. This policy contributes to keeping the product platform within a manageable scope.

Due to the nature of the product platform, customers often desire to integrate it with other software systems. CompNN offers the ‘system integration’ service to support these cases. Typical systems integrated with ProdNN are CRM-systems, help-desk systems, and call-centre systems.

5.3. The shift to the new approach

From the start of the company, ten years ago, the development process matured from a more or less ad hoc type of process to a well-defined waterfall-inspired process. This was a non iterative process with emphasis on extensive planning and up-front design. Though this process helped them to structure the development of the product platform, it became apparent that market needs and trends were not being met to their satisfaction. Customers also started to raise questions regarding close collaboration, as the products became an important part of their core business. By chance, some representatives from the company met Tom Gilb at a workshop and became interested in the concept of iterative and evolutionary development. After an introduction, they decided to try Evo in the upcoming release. At the same time, they introduced the PMT function. These two changes were not originally planned together, but experience quickly showed that they fitted well together and formed the basis for the combination of SPLE and ASD (Johansen, 2005).

6. Discussion

The integrated software processes at CompNN support three key virtues of product development. (1) Technical excellence: an open and modular platform architecture implemented using industry-standard technology enables simple development and maintenance of the product line. (2) Market knowledge and relevance: the well organized, yet nimble strategic process provides adequate decision support for company management and guidance for the development projects. (3) Agility: the adoption of Evo, and agile principles, enables fast response to changes in stakeholder requirements and accurate delivery of desired features and qualities to the users. This process configuration has been developed and matured within the organization to provide a useful balance between discipline and agility. We believe this case is an example of how to build ‘your home ground’ (Boehm and Turner, 2004) for agile SPLE.

An important observation from our findings is that CompNN’s integrated development process is not just an accidental collection of three processes. Rather, the three processes play distinct, supplemental and important roles at three different levels in CompNN’s overall software

development business. All three are lightweight to reduce waste, that is, unnecessary work (Imai, 1997). Taking a broad view of the strategic, tactical and operational processes, we see a complete process that has the same function and effects as the more general Deming improvement cycle (Deming, 2000). This is also known as the PDCA cycle, from its four main activities: *Plan* (plan how to satisfy improvement requirements); *Do* (accomplish planned actions); *Check* (monitor the actions and verify the outcome with respect to the planned effects); and *Act* (implement actions for improvement based on the acquired information). This improvement process, simple in principle, comes in many variants and is central in, for example, Total Quality Management (TQM) (Mathiassen et al., 2001). In CompNN’s case, the Plan and Act steps coincide in the strategic process where business cases, technology trends and other high-level objectives are used to create the product roadmap. Both SPLE and ASD allow and encourage continuous learning. The fact that the cycles have different durations poses no noticeable problems: strategic planning is intrinsically a long-term process. The roadmap is a stable artefact during the development of two released versions. Thus, the tactical process can be managed using the roadmap as a guide to the priorities and selection of stakeholders in the Evo process. The released products may be amended with patches during their lifetime. However, this is not a problem for the support department in charge of the operative software process, because feedback from customers is associated with the patch level of their software. Indeed, the issuing of patches enables the support department to verify whether patches actually help solve the immediate problems that customers may experience. All feedback to the support department is available for the PMT when preparing the next roadmap. In this way, the three cycles feed each other with up-to-date, accurate knowledge from the customers.

It is worth noting though, that this approach has its costs; running the strategic process and working closely with stakeholders entails a considerable extra overhead, resources that could have been invested in development (as was the previous practice). Despite this drawback, CompNN holds the benefits of these additional activities to be more valuable than spending the majority of resources purely on development activities.

Arguably, the most important benefit of this process configuration is that it helps CompNN to exploit both strategic long-term ambitions for innovation and smaller-scale tactical innovations, such as refinement of the software to meet more detailed end-user requirements. In order to implement this scheme, the PMT plays a crucial role in mediating and facilitating processes (Fægri and Hanssen, 2007). Further, the process configuration is a potent foundation for further process innovations. As long as the PDCA cycle is maintained, any component in the configuration can be further refined to improve performance. If the company experience the expected growth, the modularization gives extra organizational flexibility. For example,

introduction of separate roles for platform and product development within R&D can readily be supported because the objectives of strategic and tactical development are already defined and well established. Additionally, the explicit engagement of primarily external stakeholders in the tactical development process ensures that the company is able to supplement strategic planning by selecting stakeholders based on long-term as well as short-term interest. This is an extra benefit for use by the PMT.

The conscious engagement of market representatives is used as an important driver in the development process as a whole. It is interesting to see the differences in how this is done in practice at the strategic and tactical levels. During the development of roadmaps, knowledge obtained by PMT and CTO via any kind of market- or technology-oriented activities is exploited. In addition, key customer representatives influence the process directly through the ProdNN Advisory Board. Input is deliberately kept to a general level to establish a sufficiently wide scope. A first release is presented in detail, less detail is captured for the second, and only a rough outline is delivered for the 2-year vision, which allows for less prescriptive plans into the future. Having documented the main ambitions in the roadmap, customer representatives are given a much more detailed and direct role in the Evo projects. Selecting and involving the right stakeholders is critical however (Fægri and Hanssen, 2007). Engaged stakeholders provide requirements and feedback on a detailed level but it is the responsibility of the development projects to suggest practical solutions that will meet the quality goals stated by the engaged stakeholders. This close cooperation with a few selected stakeholders promotes valuable creativity (Hanssen and Fægri, 2006). If process innovation at CompNN had a more revolutionary style, replacing all existing practices with a pure and very formal SPLE approach, this potential could have been lost.

The present practice in CompNN was not defined and introduced from scratch as a major initiative for improving processes. However, the organization, the product platform, and the processes and their interplay, have each developed and matured over time. This has been a natural evolution, based on experience and needs (Dybå, 2000). Our overall impression from this case is that CompNN has been successful in combining and balancing practices from SPLE and ASD, essentially adopting those practices that are valuable to them, as a company.

Referring back to the figure, the strategic SPLE practices creates a fertile, controlled environment in which the tactical ASD practices can exploit the creative potential of the developers. Cockburn (2002) defines five dimensions to describe ‘an agile home ground’. These are: *size* (number of personnel), *criticality* (loss due to impact of defects), *personnel* (level of software method understanding and use), *dynamism* (percent requirement change per month) and *culture* (percent thriving on chaos versus order). The dimensions size and criticality are not significantly affected by the SPLE/ASD synergy. Looking at the personnel dimen-

sion, we see that their SPLE practices, combined with the rather disciplined IET planning tool, give the organization a rigid framework for requirement specifications. However, due to the freedom of the solution concept in Evo, developers are still encouraged and motivated to creative problem solving. Thus, the discipline enforced by SPLE and IET enables agility and creativity. Considering the dynamism dimension, much of the long-term planning is performed within the strategic SPLE process, leaving a comfortable and controlled amount of dynamism to the individual projects. Last, regarding the culture dimension, we see that a high level of order does not inhibit agility, it enables it.

7. Conclusions

The aim of this study was to describe and analyze an industrial case to understand how SPLE and ASD can be combined and to clarify associated costs and gains.

We found that these two approaches, which at first sight may seem contradicting, in fact complement each other. SPLE and ASD work together, supporting strategic and tactical objectives, respectively. At CompNN, an operational process constitutes the experience-bearing link from the tactical to the strategic process, thus completing the improvement circle. We found that this, as a whole, constitutes a framework that balances discipline and agility (Boehm and Turner, 2004).

The overall process described and discussed here involves multiple disciplines, such as product planning, knowledge management, organizational aspects, and innovation. We believe that our conclusions constitute a valuable contribution to determining the practicalities and effects of agile SPLE, being an interesting approach to industrialized software engineering.

Acknowledgements

The authors would like to thank CompNN for their time and valuable inputs. We are also grateful to Chris Wright for proofreading. This work was done as part of the EVISOFT project funded by the Research Council of Norway under Grant 156701/220.

References

- Abrahamsson, P. et al., 2002. Agile software development methods – Review and analysis. VTT Electronics, 112.
- Arisholm, E. et al., 2007. Evaluating pair programming with respect to system complexity and programmer expertise. *IEEE Transactions on Software Engineering* 33 (2), 65–86.
- Beck, K., 2000. *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- Birk, A. et al., 2003. Product line engineering, the state of the practice. *Software, IEEE* 20 (6), 52–60.
- Boehm, B., 1988. A spiral model of software development and enhancement. *IEEE Computer* 21 (5), 61–72.
- Boehm, B., 2002. Get ready for agile methods, with care. *IEEE Computer* 35 (1), 64–69.

- Boehm, B., Turner, R., 2004. Balancing Agility and Discipline – A Guide for the Perplexed. Addison-Wesley, p. 266.
- Bosch, J., 2000. Design and Use of Software Architectures – Adopting and Evolving a Product-Line Approach. Addison-Wesley.
- Bosch, J., 2006. The challenges of broadening the scope of software product families. *Communications of the ACM* 49 (12), 41–44.
- Carbon, R. et al., 2006. Integrating product line engineering and agile methods: flexible design up-front vs. incremental design. In: Workshop on Agile Product Line Engineering.
- Clements, P.C., Northrop, L., 2002. *Software Product Lines: Practices and Patterns*. Addison-Wesley.
- Cockburn, A., 2002. *Agile Software Development*. In: Cockburn, A. (Ed.), *The Agile Software Development Series*. Addison-Wesley.
- Deming, W.E., 2000. *Out of the Crisis*. The MIT Press.
- Dybå, T., 2000. Improvisation in small software organizations. *IEEE Software* 17 (5), 82–87.
- Erdogmus, H., Morisio, M., 2005. On the effectiveness of the test-first approach to programming. *IEEE Transactions on Software Engineering* (31), 3.
- Erickson, J., Lyytinen, K., Siau, K., 2005. Agile modeling, agile software development, and extreme programming: the state of research. *Journal of Database Management* 16 (4), 88.
- Fægri, T.E., Hanssen, G.K., 2007. Collaboration and process fragility in evolutionarily product development. *IEEE Software* 24 (3).
- George, B., Williams, L., 2004. A structured experiment of test-driven development. *Information and Software Technology* 46 (5 SPEC ISS), 337–342.
- Gilb, T., 2005. *Competitive engineering: A handbook for systems engineering, requirements engineering, and software engineering using P language*. Elsevier Butterworth-Heinemann, p. 480.
- Hanssen, G.K., Fægri, T.E., 2006. Agile customer engagement: a longitudinal qualitative case study. In: *International Symposium on Empirical Software Engineering (ISESE)*. Rio de Janeiro, Brazil.
- Helferich, A., Schmid, K., Herzworm, G., 2006. Product management for software product lines: an unsolved problem? *Communications of the ACM* 49 (12), 66–67.
- Imai, M., 1997. *Gemba Kaizen: A Commonsense Low-Cost Approach to Management*. McGraw-Hill.
- Johansen, T., 2005. Using evolutionary project management (Evo) to create faster, more userfriendly and more productive software. Experience report from FIRM AS. In: *International Conference on Product Focused Software Process Improvement*. Springer Verlag, Oulu, Finland.
- Käkölä, T., Dueñas, J.C., 2006. *Software Product Lines: Research Issues in Engineering and Management*. Springer Verlag.
- Keil, M., Carmel, E., 1995. Customer–developer links in software development. *Communications of the ACM* 38 (5), 33–44.
- Knauber, P. et al., 2000. Applying product line concepts in small and medium-sized companies. *Software, IEEE* 17 (5), 88–95.
- Linden, F.V.D., 2002. Software product families in Europe: The ESAPS and CAFÉ projects. In: *IEEE Software*, pp. 41–49.
- Lorge Parnas, D., Clements, P.C., 1986. A rational design process: how and why to fake it. *IEEE Transactions on Software Engineering* 12 (2), 251–257.
- Mathiassen, L., Ories-Heje, J., Ngwenyama, O., 2001. Improving software organizations: From principles to practice. In: *The Agile Software Development Series*. Addison-Wesley Professional, p. 368.
- Müller, M., Hagner, O., 2002. Experiment about test-first programming. *Software, IEE Proceedings* 149 (5), 131–136.
- Schwaber, K., Beedle, M., 2001. *Agile Software Development with Scrum*. Prentice Hall.
- Seaman, C.B., 1999. Qualitative methods in empirical studies in software engineering. *IEEE Transactions on Software Engineering* 25 (4), 557–572.
- Sugumaran, V., Park, S., Kang, K.C., 2006. Software product line engineering. *Communications of the ACM* 49 (12), 28–32.
- Tian, K., Cooper, K., 2006. Agile and software product line methods: are they so different? In: *1st International Workshop on Agile Product Line Engineering (APLE'06)*. IEEE Computer Society: Baltimore, Maryland, USA.
- Yin, R., Campbell, D.T., 2002. *Case Study Research*. Sage Publications Inc.

Geir Kjetil Hanssen works as a researcher on software process improvement and methodologies at SINTEF ICT, Norway's largest independent research institution. He has a M.Sc. degree in informatics from the University of Trondheim and 3 years of industrial experience. Currently he also holds a position as Ph.D. student at the Norwegian University of Science and Technology.

Tor Erlend Fægri works as a researcher on software process improvement and software architecture at SINTEF ICT, Norway's largest independent research institution. He has a M.Sc. degree in computing science from the University of Glasgow and 6 years of industrial experience. Currently he also holds a position as Ph.D. student at the Norwegian University of Science and Technology.